

Resumo

Este relatório técnico apresenta a arquitetura integrada de especificação, validação e automação do ecossistema OpenCode, fundamentada em quatro pilares: *Spec-Driven Development* (SDD), *Test-Driven Development* (TDD), infraestrutura de validação (*Harness*) e automação orientada a eventos (*Hooks*). A abordagem consolida princípios de Engenharia de Software – SOLID, padrões de projeto GoF, *Clean Architecture* e *Continuous Delivery* – em um pipeline fechado de qualidade que garante rastreabilidade bidirecional entre especificações e implementação. O ecossistema conta com 186 componentes documentados, 25 assertivas críticas (CAs) distribuídas em 5 especificações acadêmicas TDDAcademic, e uma arquitetura de validadores orientada a interfaces com suporte a hooks pré, pós e de erro. São discutidas as contribuições para as disciplinas de Engenharia de Software, incluindo SDD/TDD, CI/CD, arquitetura de software, padrões de projeto e verificação formal.

0.1 INTRODUÇÃO

A engenharia de software moderna demanda abordagens que integrem especificação rigorosa, validação automatizada e rastreabilidade contínua entre artefatos (SOMMERVILLE, 2011; PRESSMAN; MAXIM, 2016). O ecossistema OpenCode propõe um pipeline unificado que combina quatro camadas complementares:

1. **SDD (Spec-Driven Development):** Especificações executáveis que funcionam como contratos formais entre requisitos e implementação.
2. **TDD (Test-Driven Development):** Assertivas críticas (CAs) que validam cada aspecto da especificação.
3. **Harness:** Infraestrutura de validação que orquestra a execução dos validadores e consolida resultados.
4. **Hooks:** Sistema de eventos para automação de ações pré e pós-validação.

Este relatório documenta a arquitetura, os padrões de projeto empregados e as contribuições para as disciplinas de Engenharia de Software, apresentando o ecossistema OpenCode como estudo de caso de aplicação sistemática de princípios SDD/TDD em um ambiente multiagente.

0.2 ARCHITETURA DO PIPELINE

A arquitetura do pipeline segue o fluxo linear com retroalimentação:

$$\text{SDD} \longrightarrow \text{TDD} \longrightarrow \text{Harness} \longrightarrow \text{Hooks} \leftarrow \text{Feedback} \quad (1)$$

Cada estágio produz artefatos que alimentam o estágio subsequente, enquanto o *feedback loop* permite que métricas de cobertura e resultados de validação retroalimentem a especificação inicial.

0.2.1 SDD – Spec-Driven Development

O SDD estabelece que toda implementação deve ser precedida por uma especificação formal. No OpenCode, as especificações são documentos estruturados que definem:

- **Metadados:** Identificador, categoria, versão, autor.
- **Requisitos funcionais:** Comportamento esperado do sistema.
- **Assertivas Críticas (CAs):** Condições que devem ser satisfeitas para validação.
- **Dependências:** Relacionamentos com outras especificações.

O ecossistema mantém 186 componentes documentados com 100% de cobertura de especificações (OpenCode Team, 2025).

0.2.2 TDD – Test-Driven Development

O TDD acadêmico no OpenCode materializa-se através do sistema TDDAcademic, composto por 5 especificações e 25 Assertivas Críticas (CAs):

Tabela 1 – Especificações TDDAcademic e CAs correspondentes

Especificação	ID	CAs
TDDAcademic Base	SPEC-TDDA-BASE	CA-BASE-001 a 005
Ciclo de Desenvolvimento	SPEC-TDDA-CICLO	CA-CICLO-001 a 005
Baby Steps	SPEC-TDDA-BABY	CA-BABY-001 a 005
Regras de Refatoração	SPEC-TDDA-REGRAS	CA-REGRAS-001 a 005
Ferramentas TDD	SPEC-TDDA-FERRAMENTAS	CA-FERRAMENTAS-001 a 005

Cada CA é classificada por peso (essencial, alta, média) e domain, e inclui descrição do cenário de validação e procedimento de verificação.

0.2.3 Harness – Infraestrutura de Validação

A camada Harness é uma infraestrutura de validação composta por quatro componentes principais que orquestram o ciclo de validação:

- **DiscoveryService:** Varre o classpath para localizar validadores registrados via anotação ou convenção de nomenclatura.
- **ExecutionEngine:** Executa cada validador contra a especificação alvo, coletando resultados individuais.
- **AggregationService:** Consolida resultados parciais em um relatório unificado de validação.
- **ReportingService:** Gera artefatos de saída (JSON, texto, HTML) e dispara hooks de pós-validação.

O Harness implementa o padrão *Template Method* através da classe abstrata **AbstractValidator<T>**, que define o esqueleto do algoritmo de validação enquanto delega etapas específicas para subclasses concretas.

0.2.4 Hooks – Automação Orientada a Eventos

O sistema de hooks implementa o padrão *Observer* (GAMMA *et al.*, 1995), permitindo que agentes sejam notificados sobre eventos do pipeline:

- **Pre-Hook (on-spec-create):** Acionado antes da criação de uma especificação. Valida a estrutura, verifica consistência dos CAs e bloqueia a criação se inválida.
- **Error-Hook (on-ca-fail):** Acionado quando uma CA falha. Registra o incidente, notifica agentes responsáveis e bloqueia o avanço no pipeline.
- **Post-Hook (on-validation-complete):** Acionado após a validação completa. Gera relatório de cobertura, atualiza painel de métricas e dispara implantação se aprovado.

0.3 ARQUITETURA OOP E PADRÕES DE PROJETO

A arquitetura de validadores do OpenCode é construída sobre princípios SOLID (MARTIN, 2000) e padrões de projeto GoF (GAMMA *et al.*, 1995).

0.3.1 Hierarquia de Validadores

O núcleo da arquitetura é definido por uma interface genérica:

Listing 1 – Interface IValidator<T>

```
1 public interface IValidator<T> {  
2     ValidationResult validate(T target);  
3     boolean canHandle(Spec spec);  
4     Category getCategory();  
5 }
```

A classe abstrata `AbstractValidator<T>` implementa o padrão *Template Method*:

Listing 2 – AbstractValidator – Template Method

```
1 public abstract class AbstractValidator<T>  
2     implements IValidator<T> {  
3  
4     protected String id;  
5     protected Category category;  
6  
7     public ValidationResult validate(T target) {  
8         ValidationResult result = new ValidationResult();  
9         result.setId(this.id);  
10        if (!preValidate(target, result)) return result;  
11        doValidate(target, result);  
12        postValidate(target, result);  
13        return result;  
14    }  
15  
16    protected boolean preValidate(T t, ValidationResult r) { return true; }  
17    protected abstract void doValidate(T t, ValidationResult r);  
18    protected void postValidate(T t, ValidationResult r) {}  
19 }
```

As implementações concretas incluem:

- **SpecValidator** (SPEC-VALIDATOR-001): Valida estrutura, semântica e dependências de especificações.
- **CAValidator** (CA-VALIDATOR-001): Valida assertivas críticas individuais, seus pesos e classificações.
- **HarnessValidator** (HARNESS-VALIDATOR-001): Orquestra a execução de múltiplos validadores e gera relatórios de cobertura.

0.3.2 Padrões de Projeto Empregados

0.3.2.1 Strategy

Cada validador concreto encapsula um algoritmo de validação específico, permitindo que sejam intercambiáveis. O `HarnessValidator` seleciona dinamicamente o validador adequado com base no tipo da especificação via `canHandle()`.

0.3.2.2 Template Method

`AbstractValidator<T>` define o esqueleto do algoritmo de validação (`preValidate` → `doValidate` → `postValidate`), permitindo que subclasses redefinam etapas específicas sem alterar a estrutura geral.

0.3.2.3 Composite

O `HarnessValidator` trata validadores individuais e coleções de validadores de forma uniforme, permitindo composição hierárquica de validações.

0.3.2.4 Observer

O sistema de hooks (*Publisher*) notifica agentes registrados (*Subscribers*) sobre eventos do pipeline, implementando desacoplamento completo entre a origem do evento e os manipuladores.

0.4 INTEGRAÇÃO DO PIPELINE E CICLO DE QUALIDADE

O pipeline opera como um ciclo fechado de qualidade (*closed-loop quality*):

1. **Especificação:** Cria-se ou atualiza-se uma especificação SDD. O pre-hook valida a estrutura.
2. **Implementação:** Desenvolve-se a implementação orientada pelas especificações e CAs.
3. **Validação:** O Harness executa todos os validadores aplicáveis contra a implementação.
4. **Feedback:** Os resultados da validação retroalimentam a especificação, permitindo refinamento iterativo.
5. **Relatório:** O `ReportingService` gera artefatos de cobertura e rastreabilidade.

Este ciclo garante que toda alteração seja validada contra o conjunto completo de especificações antes de ser considerada concluída, estabelecendo um contrato formal e verificável entre requisitos e implementação (HUMBLE; FARLEY, 2010).

0.5 EXTRAÇÃO PARA AVALIAÇÃO EM BANCA

Esta seção apresenta os artefatos do OpenCode relevantes para avaliação em banca acadêmica, mapeados contra disciplinas de Engenharia de Software.

Tabela 2 – Mapeamento Artefato-Disciplina

Artefato	Disciplinas ES	Contribuição
SDD + 186 specs	SDD, Requisitos, Documentação	Rastreabilidade req.-impl.
TDDAcademic CAs	TDD, Verificação, Testes	25 assertivas críticas
Harness + Validadores	CI/CD, Arquitetura SWE-BOK	Infraestrutura de validação
Hooks + Eventos	Automação, DevOps	Gatilhos de pipeline
Interface <code>IValidator</code>	OOP, SOLID, Padrões GoF	Strategy + Template Method

Artefato	Disciplinas ES	Contribuição
AbstractValidator	Reuso, Herança, Polimorfismo	Template Method concreto
Diagrama Pipeline	Arquitetura de Software, Documentação	Visão integrada do fluxo
Feedback Loop	Melhoria Contínua, Métricas	Ciclo fechado de qualidade

0.6 CONCLUSÃO

O ecossistema OpenCode demonstra a viabilidade de integrar SDD, TDD, Harness e Hooks em um pipeline unificado de engenharia de software. As principais contribuições incluem:

1. **Arquitetura orientada a interfaces** com separação clara de responsabilidades e adesão aos princípios SOLID.
2. **Pipeline de qualidade fechado** que garante rastreabilidade bidirecional entre especificações e implementação.
3. **Sistema de hooks extensível** que permite automação de ações pré e pós-validação sem acoplamento.
4. **Documentação completa** com 186 componentes e 100% de cobertura de especificações.
5. **Estrutura de validação multiagente** com suporte a composição hierárquica de validadores.

Os padrões de projeto empregados – Strategy, Template Method, Composite e Observer – fornecem uma base sólida para extensão futura do ecossistema, enquanto o ciclo fechado de qualidade estabelece um arcabouço replicável para projetos que exigem rigor entre especificação e implementação.

0.7 TRABALHOS FUTUROS

Entre as direções para trabalho futuro, destacam-se:

- Integração com ferramentas formais de verificação (Z3, Coq) para validação simbólica de especificações.
- Geração automática de casos de teste a partir das especificações SDD utilizando técnicas de *model-based testing*.
- Expansão do sistema de hooks para suportar encadeamento de eventos e orquestração multiagente.
- Métricas quantitativas de eficácia do pipeline em projetos reais.
- Publicação do ecossistema como *open-source* para replicação e validação pela comunidade acadêmica.

Referências

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. [S.l.]: Addison-Wesley, 1995.

HUMBLE, J.; FARLEY, D. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. [S.l.]: Addison-Wesley, 2010.

MARTIN, R. C. Design principles and design patterns. *Object Mentor*, 2000.

OpenCode Team. *OpenCode: Intelligent Software Engineering at the Command Line*. 2025.
<<https://opencode.ai/>>.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. [S.l.]: AMGH, 2016.

SOMMERVILLE, I. *Engenharia de Software*. 9. ed. [S.l.]: Pearson, 2011.